

Discussion 9: Queueing Theory & Distributed (file)systems

April 24, 2026

Contents

1	Queueing Theory	2
1.1	Little's law.	2
1.2	M/M/1 queues.	3
2	Reliability	5
3	Transactions	6
3.1	Concept Check	6
4	Journaling	8
5	Distributed Data Processing	9
5.1	Concept Check	9
5.2	Leetcode golf	9
6	Distributed Systems	10
6.1	Concept Check	10
6.2	Two Phase Commit	10

1 Queueing Theory

In this question, we'll derive the formulas for an M/M/1 queue, and think about the properties as it relates to a real-world system. First, we will define the notation for a simple queueing system.

Let

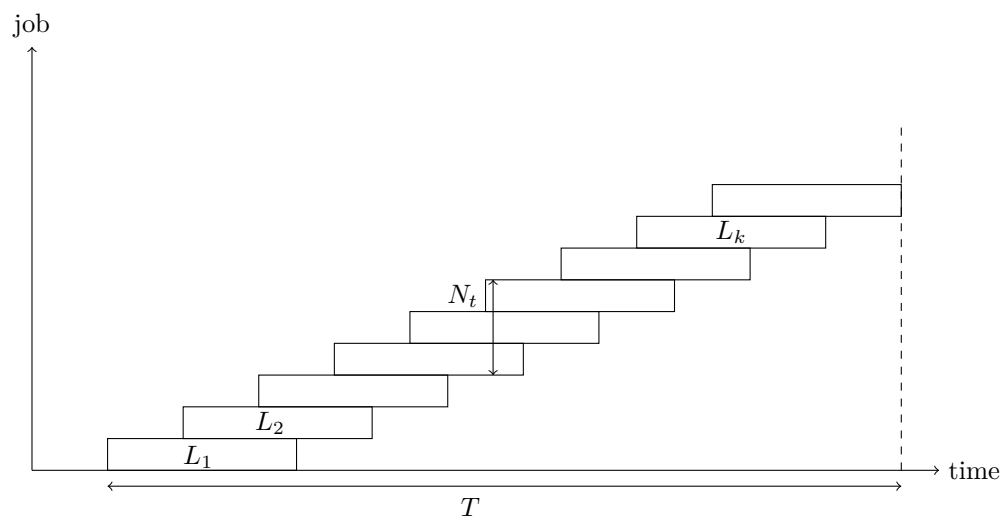
- λ be the arrival rate
- μ be the mean service rate
- T_q be the average time in queue
- T_s be the average service time

1.1 Little's law.

For this first part, we will derive Little's law. Recall that Little's law states that

$$\bar{N} = \bar{\lambda} \cdot \bar{L}$$

i.e., the average number of jobs in the system, $\bar{N}$ is the product of the average arrival rate $\bar{\lambda}$ and the average response time $\bar{L}$.

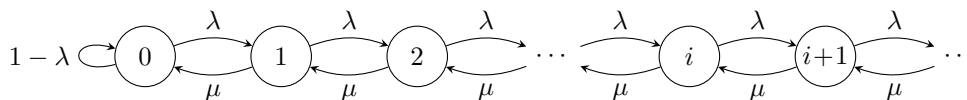


1. Let L_i be the response time for job i . Write $\bar{N}$ in terms of $\sum L_i$ and T .

2. Derive Little's law, $\bar{N} = \bar{\lambda} \cdot \bar{L}$. It may help to introduce a factor of N .

1.2 M/M/1 queues.

Recall that the "M" in M/M/1 stands for Markovian, or memoryless. We say this as the arrivals and services do not remember past events, and therefore can be modelled as an exponential distribution. Due to this memoryless property, we can model the number of jobs in the system as an infinite birth-death chain, where the forward transition probability (the arrival rate) of λ and the backward transition probability (the service rate) of μ .



Additionally, recall that when $\lambda < \mu$, this Markov chain has a stationary distribution. We can compute this stationary distribution by satisfying the local balance equations for a reversible chain,

$$\pi(i)P(i, i+1) = \pi(i+1)P(i+1, i)$$

1. What does state i represent? What about $\pi(i)$?

2. Let $\rho = \frac{\lambda}{\mu}$ be the system utilisation. Show that $\pi(0) = 1 - \rho$, and identify the distribution of $\pi(i)$.

3. What does the system look like when $\rho < 1$ is small? What does this look like in terms of the "burstiness" of the CPU?

4. What about when $\rho \rightarrow 1$? What happens to the mean length of the queue in this case?

5. Let L be the number of jobs in the system. Show that

$$\mathbb{E}[L] = \frac{\rho}{1 - \rho}$$

You may use the fact that $\sum_{n=0}^{\infty} n\rho^n = \frac{\rho}{(1-\rho)^2}$.

6. We can write $\mathbb{E}[L] = \mathbb{E}[L_q] + \mathbb{E}[L_s]$, the sum of the number of jobs queued, and the number of jobs *currently being serviced*. Show that

$$\mathbb{E}[L_q] = \frac{\rho^2}{1 - \rho}$$

and apply Little's law to show that the average queueing time

$$\mathbb{E}[T_q] = \mathbb{E}[T_s] \cdot \frac{\rho}{1 - \rho}$$

7. Explain why when $\rho \approx 1$, small increases in load lead to large increases in the delay. How does this apply to real world systems?

8. We also saw that in a real-world system, queueing delay can grow unboundedly when utilisation is not 1. Why is this?

9. In contrast, let's consider a deterministic system with regular arrivals. How does this differ from the previous part? Can we achieve $\rho = 1$?

2 Reliability

Availability

Availability is the probability that the system can accept and process requests. This is measured in “nines” of probability (e.g. 99.9% is said to be “3-nines of availability”).

Durability

Durability is the ability of a system to recover data despite faults (i.e. fault tolerance). It’s important to note that durability does not necessarily imply availability.

When making a file system more durable, there are multiple levels which we need to concern ourselves with. For small defects in the hard drive, Reed-Solomon error correcting codes can be used in each disk block. When using a buffer cache or any other delayed write mechanism, it’s important to make sure dirty data gets written back to the disk. To combat unexpected failures or power outages, the computer can be built with a special, battery-backed RAM called non-volatile RAM (NVRAM) for dirty blocks in the buffer cache.

To make sure the data survives in the long term, it needs to be replicated to maximize the independence of failures. **Redundant Array of Inexpensive Disks (RAID)** is a system that spreads data redundantly across multiple disks in order to tolerate individual disk failures. RAID 1 will **mirror** a disk onto another “shadow” disk. Evidently, this is a very expensive solution as each write to a disk actually incurs two physical writes. RAID 5 will stripe data across n multiple disks to allow for a single disk failure. Each stripe unit consists of $n - 1$ blocks and one **parity block**, which is created by XOR-ing the $n - 1$ blocks. To recover from a disk failure, the system simply needs to XOR the remaining blocks.

Reliability

Reliability the ability of a system or component to perform its required functions under stated conditions for a specified period of time. This means that the system is not only up (i.e. availability), but also performing its jobs correctly. Reliability includes the ideas of availability, durability, and security.

One approach taken by FAT and FFS (in combination with **fsck**) is **careful ordering and recovery**. For instance, creating a file in FFS may be broken down into the following steps

1. Allocate data block. Write data block.
2. Allocate inode. Write inode block.
3. Update free map. Update directory entry. Update modify time for directory entry.

To recover from a crash, **fsck** might take the following steps.

1. Scan inode table.
2. If any unlinked files (not in any directory), delete or put in lost and found directory.
3. Compare free block bitmap against inode trees.
4. Scan directories for missing update/access times.

It’s important to note that this is not a foolproof method. While there are a few ways that failures can happen in spite of this method, the file system will be recoverable most of the times.

The other approach is to use **copy on write (COW)**. Instead of updating data in-place, new versions are written to a new location on disk, and the appropriate mappings and references to these data are subsequently updated. Since the mappings and references are updated last, this allows for easy recovery if the system crashes sometime in the middle of updating data since the old data and mapping will still be in tact. Furthermore, data is being only added, not modified, so batch updates and parallel writes can help improve performance.

3 Transactions

A more general way to handle reliability is through the use of **transactions**, indivisible units which execute independently from other transactions. Transactions typically follow the **ACID properties**.

Atomicity A transaction must occur in its entirety or not at all.

Consistency A transaction takes system from one consistent state (i.e. meets all integrity and correctness constraints) to another.

Isolation Each transaction must *appear* to execute on its own.

Durability A committed transaction's changes must persist through crashes.

The idea of a transaction is similar to that of a critical section with the newly added constraint of durability. Each entry in a transaction needs to be **idempotent**, which have the same effect when executed once or many times (i.e. $f(f(x)) = f(x)$).

Transactions are recorded on **logs/journals** which are stored on disk for persistence. Writes to a Log are assumed to be atomic. Logs will typically use a circular buffer as its data structure, which maintains a head and tail pointer.

Transactional file systems use the idea of transactions and logs to make the file system more reliable. There are two main types of transactional file systems: journaling and log structured.

Journaling file systems use **write-ahead logging (WAL)** where log entries are written to disk *before* the data gets modified. During the preparation phase, all planned updates are appended to the log. Each log entry is tagged with the transaction id. During the commit phase, a commit record is appended to the log, indicating the transaction must happen at some point. Next, the write back will take place *asynchronously* after the commit phase where the transaction's changes will be applied to persistent storage. In the background, garbage collection will take place to clean up fully written back transactions. When recovering from a crash, only the committed transactions are replayed to restore the state of the file system.

Log structured file systems (LFS) use the log as the storage. The log becomes one contiguous sequence of blocks that wrap around the whole disk.

3.1 Concept Check

1. Why does each entry in a transaction need to be idempotent?

2. Consider a file system with a buffer cache. Your program creates a file called `pintos.bean`. While the changes have been logged with a commit entry, the tail of the log still points to before the start of the log entry corresponding to creating `pintos.bean`. Assuming no further disk reads or writes have happened, if your program wants to read `pintos.bean`, will it need to scan through the logs?

3. What is the purpose of a commit entry in a log?

4 Journaling

You create two new files, F_1 and F_2 , right before your laptop's battery dies. You plug in and reboot your computer, and the operating system finds the following sequence of log entries in the file system's journal.

1. Find free blocks $x_1, x_2, \dots, x_n$ to store the contents of F_1 , and update the free map to mark these blocks as used.
2. Allocate a new inode for the file F_1 , pointing to its data blocks.
3. Add a directory entry to F_1 's parent directory referring to this inode.
4. *Commit*
5. Find free blocks $y_1, y_2, \dots, y_n$ to store the contents of F_2 , and update the free map to mark these blocks as used.
6. Allocate a new inode for the file F_2 , pointing to its data blocks.

You may assume a single write to disk is an atomic operation.

1. What are the possible states of files F_1 and F_2 *on disk* at boot time?

2. Say the following entries are also found at the end of the log.
 7. Add a directory entry to F_2 's parent directory referring to F_2 's inode.
 8. *Commit*

How does this change the possible states of file F_2 on disk at boot time?

3. Say the log contained only entries (5) through (8) shown above. What are the possible states of file F_1 on disk at the time of the reboot?

4. When recovering from a system crash and applying the updates recorded in the journal, does the OS need to check if these updates were partially applied before the failure?

5 Distributed Data Processing

5.1 Concept Check

1. True or False: Completed map tasks are re-executed when their worker crashes.

2. True or False: User tasks must be idempotent for MapReduce.

3. How does a reduce worker find and read the relevant KV pairs on workers' disks?

4. What are some performance benefits of lazily evaluating transformations?

5.2 Leetcode golf

Suppose we downloaded a file of all solution statistics from Leetcode. Each file is a comma-separated list of entries formatted as {key: PROBLEM_NAME, value: CODE_LENGTH} (e.g. {"2SUM", 162}). Implement the following MapReduce program to find the code golf champion for each problem!

Note: Code golfing is a competition to solve a given problem with the shortest source code possible.

```
map(key problem_name, value code_length) {  
    Emit(_____, _____);  
}  
  
reduce(key problem_name, List<value> length_lists) {  
    lowest = _____;  
    for (value length : length_lists) {  
        _____;  
    }  
    Emit(_____, _____);  
}
```

6 Distributed Systems

6.1 Concept Check

1. The vanilla implementation of 2PC logs all decisions. How could 2PC be optimized to reduce logging?

2. 2PC exhibits blocking behavior where a worker can be stalled until the coordinator recovers. Why is this undesirable?

6.2 Two Phase Commit

Consider a system with one coordinator (C) and three workers (W_1 , W_2 , W_3). The following latencies are given for each worker.

Worker	Send/Receive (each direction)	Log
W_1	400 ms	10 ms
W_2	300 ms	20 ms
W_3	200 ms	30 ms

You may assume all other latencies not given are negligible. C has a timeout of 3 s, log latency of 5 ms, and can communicate with all workers in parallel.

1. What is the minimum amount of time needed for 2PC to complete successfully?

2. Consider that all three workers vote to commit during the preparation phase. The coordinator broadcasts a commit decision to all the workers. However, W_2 crashes and does not recover until immediately after the coordinator's timeout phase. Does this transaction commit or abort? What is the latency of this transaction, assuming no further failures?

